

Proyecto Final Capstone

Universidad Tecnológica Nacional

Tecnicatura Universitaria en Programación

Materia: Arquitectura y Sistemas Operativos

Profesor: Menvielle Mateo

Alumno: Stemphelet Tobías

Fecha: 10 DE JULIO DE 2026

En el presente proyecto final Capstone se va a analizar e implementar el proceso completo de como consumir datos de un dispositivo IoT mediante una API REST para procesarlos con la plataforma Node - RED y visualizarlo en tiempo real. Todo el sistema fue contenerizado con Docker, utilizando todas las herramientas y conceptos abordados en la materia Arquitectura y Sistemas Operativos.

A lo largo del presente informe se detallan los fundamentos teóricos de las tecnologías utilizadas (Docker, API REST, IoT y Node - RED), la arquitectura del sistema implementado mediante un diagrama de bloques, el desarrollo técnico del flujo de datos, los resultados obtenidos y las dificultades encontradas durante el proceso.

¿Qué es Docker y para que se utiliza?

Docker es una plataforma de código abierto que permite la automatización del despliegue de aplicaciones dentro de contenedores de software, aislado del sistema operativo con todas sus configuraciones específicas y dependencias necesarias para un funcionamiento óptimo del mismo. A diferencia de las máquinas virtuales, Docker interactúa con el Kernel del sistema operativo anfitrión, lo que los hace más ligeros y eficientes a la hora de usar recursos del equipo.

Se utiliza principalmente para probar y ejecutar aplicaciones de forma segura, garantizando que funcionen de la misma manera en cualquier computadora o servidor. Además, facilita el trabajo en equipo simplificando la instalación de programas y optimiza los recursos al permitir ejecutar múltiples aplicaciones de forma independiente, sin que estas interfieran entre sí.

Gracias a todas estas características, Docker es una herramienta fuertemente utilizada en el desarrollo de software y en empresas tecnológicas para mejorar la eficiencia y la portabilidad de las aplicaciones.

¿Qué es una API REST y cómo se consulta?

Una Api Rest es una interfaz que permite la comunicación de diferentes aplicaciones a través de Internet, su función es facilitar el intercambio de información entre clientes (aplicación o sitio web) y un servidor, utilizando un protocolo HTTP.

La API REST se consulta mediante HTTP dirigidas por una dirección URL o Endpoint, cada consulta usa un método según la acción que se desea realizar. GET para obtener información, POST para crear nuevos datos, PUT para modificar datos existentes y DELETE para eliminar datos. Las consultas pueden realizarse utilizando herramientas como un navegador web o directamente desde un programa mediante código. El servidor procesa la solicitud y devuelve una respuesta, generalmente en formato **JSON**, que contiene los datos solicitados o el resultado de la operación realizada.

Características generales de un sistema IoT y tipos de datos que genera.

Un sistema IoT (Internet de las cosas) es un conjunto de dispositivos físicos conectados a internet que recopilan, envían y reciben información de manera automática. Estos dispositivos tienen incorporado sensores que les permiten interactuar con el entorno y comunicarse con otros sistemas.

Características principales:

- Conectividad a Internet
- Recolección de datos en tiempo real
- Automatización de procesos
- Monitoreo remoto
- Capacidad de intercambiar información entre distintos dispositivos a distancia

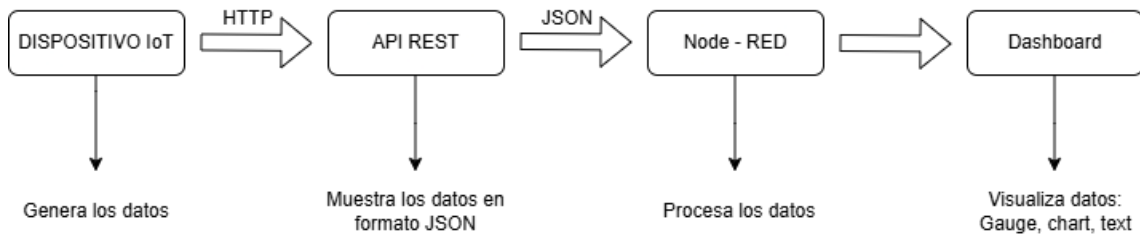
Los datos que genera un sistema IoT dependen de su aplicación, pero generalmente incluyen mediciones de sensores como temperatura, humedad, presión, ubicación (GPS), velocidad, consumo de energía, entre otros. Estos datos se utilizan para analizar el funcionamiento del sistema y en base a eso tomar decisiones para mejorar la eficiencia de los recursos existentes.

Función de Node-RED en una arquitectura IoT :

La plataforma Node-RED cumple el papel de integrar, procesar y automatizar la comunicación entre dispositivos IoT con las aplicaciones o servicios que utilizan la información compartida. Permite recibir datos que provienen de sensores para procesarlos, editarlos y enviarlos a otros dispositivos, bases de datos o servicios web.

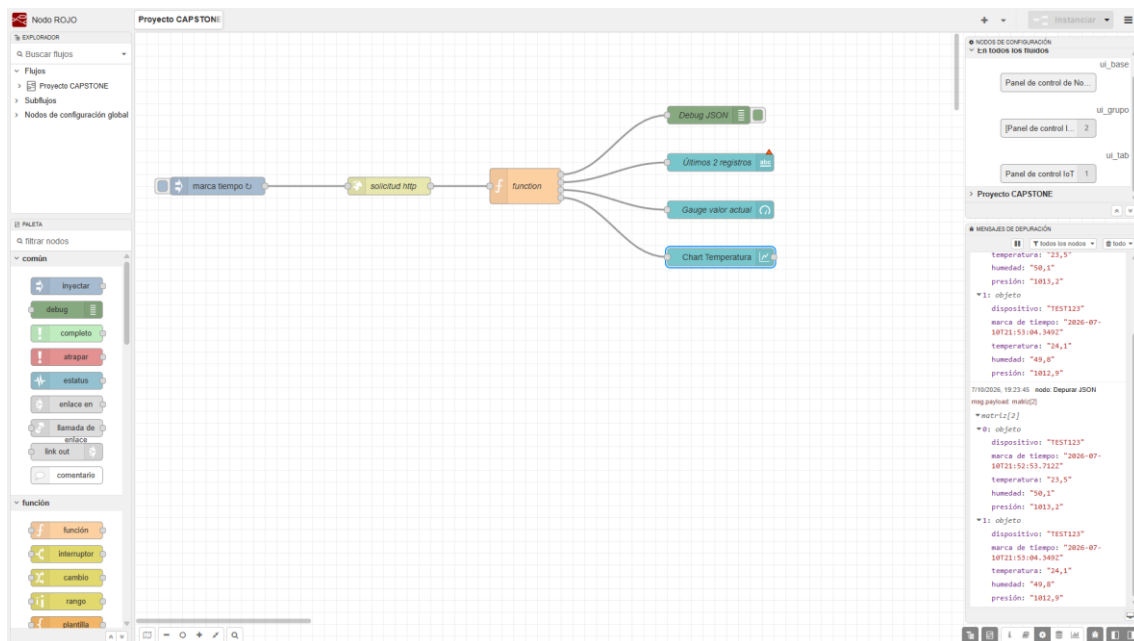
Su principal ventaja es que utiliza una interfaz gráfica basada en nodos, lo que facilita el desarrollo de flujos de trabajo sin necesidad de escribir grandes cantidades de código. De esta manera, Node-RED simplifica la creación de aplicaciones IoT, la automatización de tareas y el monitoreo de dispositivos conectados.

Diagrama de bloques del sistema IoT:

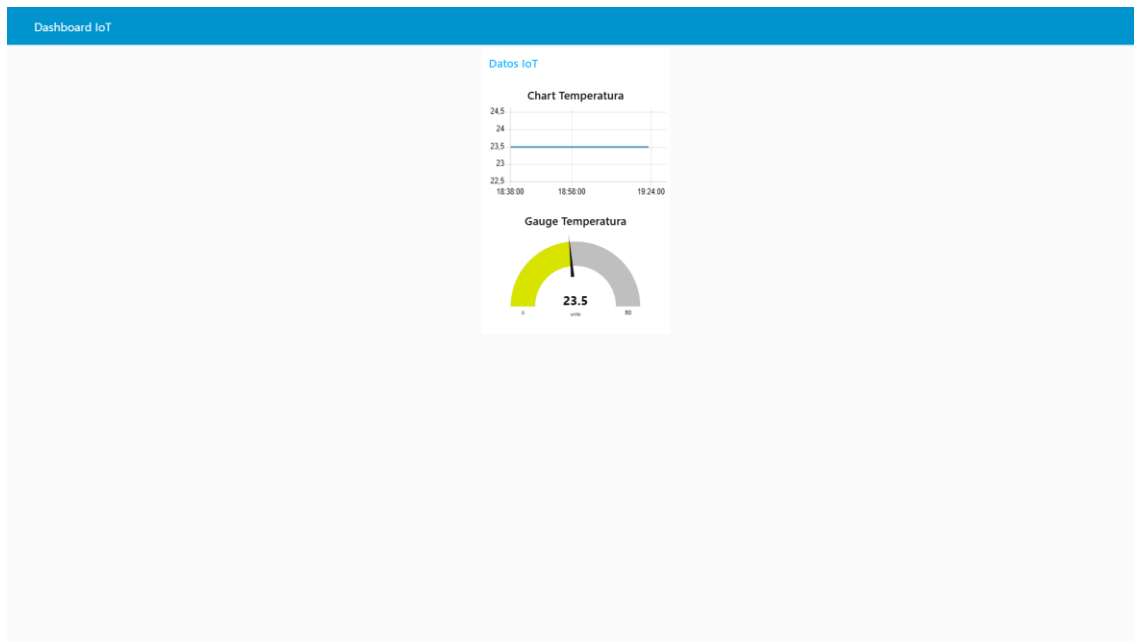


Se ha implementado un flujo Node – Red compuesto por los siguientes nodos: HTTP request, que realiza la consulta del Endpoint cada 10 segundos mediante un nodo Inject que configurado con un intervalo repetitivo. Luego se utilizó el Nodo Function que ordena los registros recibidos y extrae los dos más recientes y por último se usaron nodos como chart, gauge y text para visualizar los valores obtenidos en tiempo real del dashboard.

Captura del flujo Node – RED:



Captura del Dashboard:



Captura del JSON RECIBIDO:

```
formato al texto
{
  "device": "FE51121",
  "timestamp": "2020-07-10T21:38:05.894Z",
  "temperature": "23.5",
  "humidity": "56.3",
  "pressure": "1011.2"
},
{
  "device": "FE51121",
  "timestamp": "2020-07-10T21:39:51.743Z",
  "temperature": "24.1",
  "humidity": "49.0",
  "pressure": "1012.0"
},
{
  "device": "FE51121",
  "timestamp": "2020-07-10T21:40:11.824Z",
  "temperature": "23.6",
  "humidity": "48.2",
  "pressure": "1011.5"
}
```

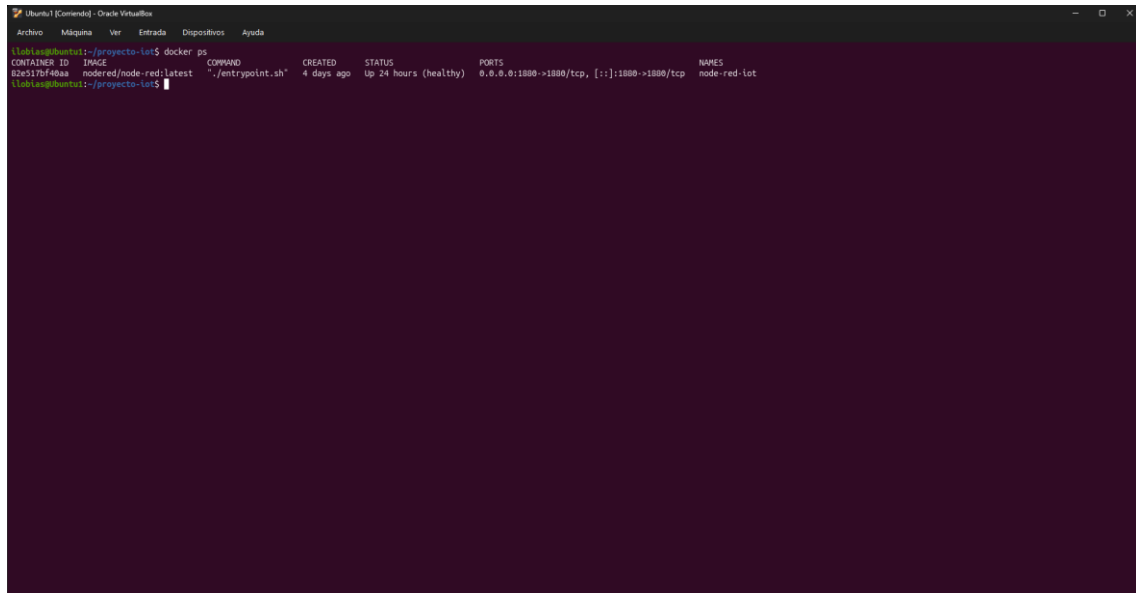
El sistema se contenerizó mediante un archivo `docker-compose.yml`, que automatiza la creación del entorno necesario para ejecutar Node-RED. El archivo especifica:

- **La imagen del servicio:** `nodered/node-red`, la imagen oficial de Node-RED.
- **El mapeo de puertos:** `1880:1880`, que expone el puerto interno del contenedor hacia el mismo puerto en el host, permitiendo acceder al editor y al dashboard desde <http://localhost:1880>.

- **Un volumen persistente:** node_red_data, que almacena los flujos, la configuración y las paletas instaladas fuera del contenedor, garantizando que los datos no se pierdan si el contenedor se reinicia o se recrea.

Gracias a esta configuración, todo el entorno puede levantarse con un único comando en Ubuntu (docker compose up -d), asegurando la portabilidad y reproducibilidad del sistema en cualquier equipo con Docker instalado.

Captura del contenedor en ejecución:



```
Ubuntu [Comando] - Oracle VM VirtualBox
Archivo  Máquina  Ver  Entrada  Dispositivos  Ayuda

root@esplubuntu:~/proyecto-lot$ docker ps
CONTAINER ID   IMAGE                COMMAND                  CREATED        STATUS              PORTS                               NAMES
8283176f40aa   nodered/node-red:latest  ./entrypoint.sh         4 days ago    Up 24 hours (healthy)  0.0.0.0:1880->1880/tcp, [::]:1880->1880/tcp  node-red-lot

root@esplubuntu:~/proyecto-lot$
```

Durante el desarrollo del proyecto se presentaron algunas dificultades con respecto a la configuración de la red de contenedores, el código del nodo function y la muestra de los datos recibidos de la API en el flujo Node - RED.

Pero todo esto dio la oportunidad de adquirir nuevos conocimientos y reforzar conceptos vistos en la materia como la gestión de comunicación entre procesos, servidores, redes y la virtualización de contenedores, permitió integrar de forma práctica los conocimientos teóricos adquiridos como Docker, Node-RED y comandos de Ubuntu como herramientas modernas del desarrollo de software para el despliegue de arquitecturas y datos en tiempo real.

Bibliografía:

Red Hat: <https://www.redhat.com/es/topics/api/what-is-a-rest-api>

Aws: <https://aws.amazon.com/es/what-is/iot/>

Aws: <https://aws.amazon.com/es/docker/>